

0

12

25

37

50

62

75

DOCUMENTO TÉCNICO · V1.4.0

Sonda

Banco de dados

Diagrama de relacionamentos, estruturas e formato binário que o índice invertido do motor de busca deve seguir, e os dados que circulam fora dele.

CARTA DO PROJETO

PROJETO	Sonda · busca de texto completo
DOCUMENTO	Banco de dados
VERSÃO	1.4.0
DATA	Outubro de 2026
AUTOR	Erik L. Pimenta
LICENÇA	MIT

14_2

31

38

Sumário

01 **Visão geral** 4

Por que não um banco relacional 4

Princípios 4

Convenções 4

Os arquivos 5

02 **Diagrama de relacionamentos** 6

03 **Relacionamentos** 7

Chaves e cardinalidades 7

Chave do documento 7

04 **Estruturas** 9

FONTE 9

EXECUCAO_INDEXACAO 9

PONTO_DE_GRAVACAO 9

SEGMENTO 10

CAMPO 10

DOCUMENTO 10

TERMO 11

POSTAGEM e POSICAO 11

BLOCO_DE_SALTO 12

NORMA 12

VALOR_DOCUMENTO 12

EXCLUSAO 12

05 **Formato binário** 14

Cabeçalho e rodapé 14

Dicionário de termos (.tim e .tip) 14

Postagens (.doc e .pos) 14

Campos armazenados (.fdt e .fdx) 15

Normas, doc values, exclusões e ponto de gravação 15

06 **Regras de consistência** 16

Gravação (commit) 16

Exclusões e atualizações 16
Mesclagem 16
Leitura concorrente 16

07 **Dados fora do índice** 17
ESTATISTICA_BUSCA 17
EVENTO_WEBHOOK 17
Formatos de exportação 18

08 **Como criar e manter** 19
Criando o índice 19
Backup e restauração 19
Evoluindo o formato 19

CAPÍTULO 01

Visão geral

A Sonda não guarda os documentos num banco SQL e não usa `LIKE '%texto%'`. O "banco de dados" do motor de busca é o próprio índice invertido, gravado em arquivos binários na pasta `dados/indice`, mais um catálogo em JSON (`dados/catalogo.json`) com as fontes e o histórico de indexação.

Este documento descreve o modelo lógico desse banco (entidades, chaves e relacionamentos), o formato físico de cada arquivo e as regras que mantêm tudo consistente mesmo com indexação paralela, quedas de energia e mesclagens em segundo plano. A versão 1.4.0 não muda o formato do índice: segmentos gravados pelas versões anteriores continuam sendo lidos sem reindexar. O capítulo 7 descreve os dados novos que circulam fora do índice (estatísticas de busca, webhooks e exportações).

1.1 Por que não um banco relacional

Um banco relacional responde bem a "qual linha tem o id 42", mas não a "quais documentos falam de índices invertidos, do mais relevante ao menos relevante". Um `LIKE` com curinga no início impede o uso de índices B-tree e obriga a ler todas as linhas, não entende plural, acento nem radical, e não ordena por relevância.

O índice invertido inverte a relação: para cada termo, guarda a lista ordenada dos documentos em que ele aparece. Uma consulta lê só as listas dos termos pesquisados, que já vêm comprimidas e ordenadas para serem cruzadas rapidamente.

1.2 Princípios

- **Segmentos imutáveis:** um segmento nunca é alterado depois de gravado. Inclusões criam segmentos novos; exclusões ficam num mapa de bits à parte.
- **Ponto de gravação atômico:** o arquivo `segments_N` lista os segmentos válidos. Ele é gravado como `pending_segments_N`, sincronizado no disco e renomeado. Quem abre o índice lê sempre o maior N válido.
- **Checksum em todo arquivo:** cada arquivo termina com um rodapé de 16 bytes com CRC32 de todo o conteúdo anterior.
- **Um escritor, muitos leitores:** o arquivo `write.lock` garante um único processo gravando; leitores abrem os arquivos com compartilhamento de leitura e exclusão e não bloqueiam ninguém.
- **Ordem por bytes UTF-8:** termos são ordenados pela sequência de bytes UTF-8, o que permite busca binária e prefixos sem depender de cultura.

1.3 Convenções

TIPO	TAMANHO	DESCRIÇÃO
`byte`	`1 byte`	Inteiro sem sinal de 0 a 255
`int32` / `int64`	`4 / 8 bytes`	Inteiros little-endian
`vint`	`1 a 5 bytes`	Inteiro de tamanho variável: 7 bits por byte, o bit alto indica que há mais bytes

TIPO	TAMANHO	DESCRIÇÃO
`vlong`	1 a 9 bytes	Mesma codificação do vint para 64 bits, só para valores não negativos
`zlong`	1 a 10 bytes	Inteiro com sinal em zigue-zague (0, -1, 1, -2...) e depois vlong
`texto`	vint + bytes	Tamanho em bytes seguido do conteúdo em UTF-8
`delta`	vint	Diferença para o valor anterior da mesma lista
`bloco FOR`	1 + 16 × bits	128 inteiros empacotados com a largura em bits do maior valor (frame of reference)

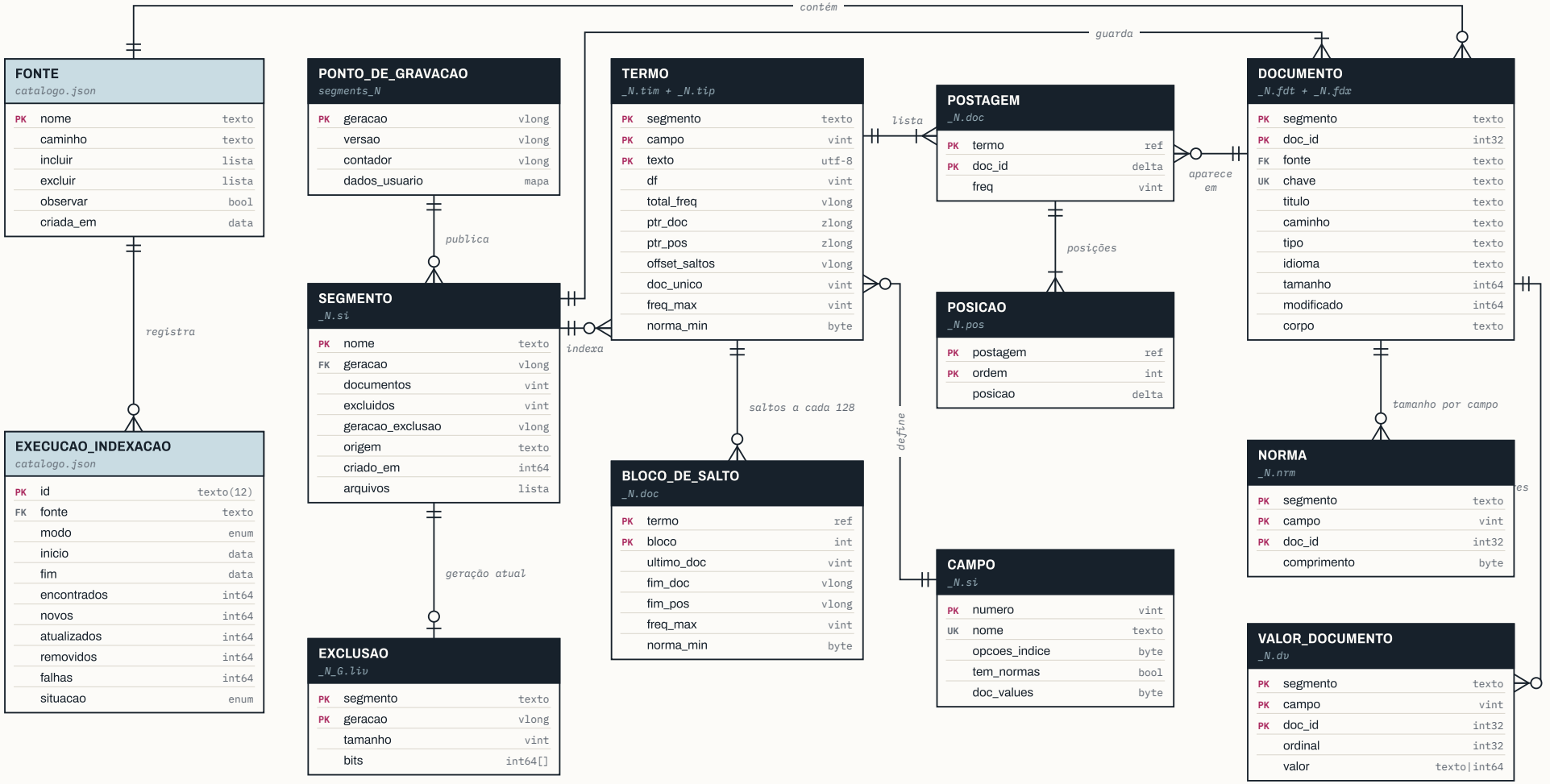
1.4 Os arquivos

ARQUIVO	ENTIDADE	CONTEÚDO
`segments_N`	PONTO_DE_GRAVACAO	Geração, versão, contador de nomes e a lista de segmentos com suas exclusões
`_N.si`	SEGMENTO, CAMPO	Nome, quantidade de documentos, origem, data, campos e lista de arquivos do segmento
`_N.tip`	TERMO (índice)	Primeiro termo de cada bloco de 32 termos e o ponteiro do bloco, carregados em memória
`_N.tim`	TERMO (dicionário)	Todos os termos de cada campo, com prefixo compartilhado e metadados
`_N.doc`	POSTAGEM, BLOCO_DE_SALTO	Listas de documentos e frequências, em blocos FOR de 128 e cauda em vint, mais a tabela de saltos
`_N.pos`	POSICAO	Posições de cada ocorrência, em delta, para consultas de frase
`_N.nrm`	NORMA	1 byte por documento e por campo com o tamanho do campo quantizado
`_N.fdt` / `_N.fdx`	DOCUMENTO	Campos armazenados em blocos comprimidos com Brotli e o índice dos blocos
`_N.dv`	VALOR_DOCUMENTO	Valores por documento para facetas e ordenação: tipo, idioma, fonte, data e tamanho
`_N_G.liv`	EXCLUSAO	Mapa de bits dos documentos excluídos na geração G
`write.lock`	—	Trava de escrita exclusiva, apagada ao fechar
`catalogo.json`	FONTE, EXECUCAO_INDEXACAO	Fontes cadastradas pelo painel e as últimas 100 execuções (fica em `dados/`)

CAPÍTULO 02

Diagrama de relacionamentos

Entidades com cabeçalho escuro são gravadas pelo motor no índice; as de cabeçalho azul-claro ficam no catálogo JSON. O nome do arquivo físico aparece sob o nome da entidade. **PK** é chave primária, **FK** chave estrangeira e **UK** valor único. Notação pé de galinha: o traço duplo indica exatamente um, o círculo indica zero, o traço simples indica pelo menos um e o pé de galinha indica muitos.



CAPÍTULO 03

Relacionamentos

Cada relacionamento abaixo é garantido pelo próprio formato dos arquivos, não por restrições de um banco externo.

3.1 Chaves e cardinalidades

DE	PARA	CARDINALIDADE	COMO É GARANTIDO
FONTE	EXECUCAO_INDEXACAO	1 : 0..N	Cada execução grava o nome da fonte; o catálogo mantém as 100 mais recentes
FONTE	DOCUMENTO	1 : 0..N	A chave do documento começa com `fonte:`, e o campo `source` é indexado como palavra-chave
PONTO_DE_GRAVACAO	SEGMENTO	1 : 0..N	`segments_N` lista o nome, a quantidade de documentos e a geração de exclusão de cada segmento
SEGMENTO	DOCUMENTO	1 : 1..N	Documentos são numerados de 0 a documentos – 1 dentro do segmento (doc_id local)
SEGMENTO	TERMO	1 : 0..N	Cada segmento tem dicionário próprio, separado por campo
SEGMENTO	EXCLUSAO	1 : 0..1 por geração	Só a geração citada no ponto de gravação vale; as antigas são apagadas
CAMPO	TERMO	1 : 0..N	O dicionário é dividido em seções por número de campo, em ordem crescente
TERMO	POSTAGEM	1 : 1..N	Os metadados do termo apontam o início da lista em `.doc`; df diz quantas postagens ler
DOCUMENTO	POSTAGEM	1 : 0..N	A postagem guarda o doc_id em delta; ela só existe se o documento contiver o termo
POSTAGEM	POSICAO	1 : 1..N	Cada postagem tem exatamente freq posições em `.pos`
TERMO	BLOCO_DE_SALTO	1 : 0..N	Um bloco de salto a cada 128 postagens; listas menores não têm tabela
DOCUMENTO e CAMPO	NORMA	1 : 1 por campo	Título, corpo e caminho têm 1 byte por documento
DOCUMENTO e CAMPO	VALOR_DOCUMENTO	1 : 1 por campo	Tipo, idioma e fonte por ordinal; modificado e tamanho como inteiro

3.2 Chave do documento

Todo documento tem uma chave única no formato `fonte:caminho/relativo.ext`, por exemplo `exemplos:tecnologia/indice-invertido.md`. Ela é indexada no campo `id` sem análise de texto. Para atualizar ou excluir, o escritor procura essa chave no dicionário de cada segmento e marca o doc_id encontrado no mapa de

exclusões.

O doc_id é local ao segmento e muda quando segmentos são mesclados. Por isso a API, o painel, as exportações e os webhooks identificam documentos sempre pela chave, nunca pelo doc_id. Nas rotas da API, a chave viaja em Base64 URL (`/api/documentos/{chave}`, `/semelhantes` e `/texto`).

CAPÍTULO 04

Estruturas

Campos, tipos e regras de cada entidade do diagrama.

4.1 FONTE

dados/catalogo.json e Sonda:Sources no appsettings

Pasta varrida pelo indexador. Fontes do appsettings são somente leitura no painel; as criadas pela API ficam no catálogo.

CAMPO	TIPO	CHAVE	DESCRIÇÃO
`nome`	texto (até 40)	PK	Letras minúsculas, números, hífen e sublinhado. Prefixo de todas as chaves de documento
`caminho`	texto		Pasta absoluta no servidor
`incluir`	lista de padrões		Padrão glob, por exemplo `*.md`. Padrão: txt, md, markdown, html e htm
`excluir`	lista de padrões		Padrão glob, por exemplo `**/node_modules/**`
`observar`	bool		Liga o FileSystemWatcher e reindexa a fonte alguns segundos depois de cada alteração
`criada_em`	data UTC		Momento do cadastro

O caminho precisa existir e, se `Sonda:AllowedRoots` estiver preenchido, estar dentro de uma das raízes permitidas.

4.2 EXECUCAO_INDEXACAO

dados/catalogo.json

Resumo de cada varredura de uma fonte, incremental ou completa. Cada execução concluída também dispara o evento `execucao` dos webhooks (capítulo 7).

CAMPO	TIPO	CHAVE	DESCRIÇÃO
`id`	texto(12)	PK	Identificador aleatório
`fonte`	texto	FK	FONTE.nome
`modo`	Incremental | Full		Incremental pula arquivos com mesma data e tamanho; completa reprocessa tudo
`inicio`, `fim`	data UTC		Duração da execução
`encontrados`, `novos`, `atualizados`, `removidos`, `falhas`	int64		Contadores da execução
`situacao`	texto		`concluida`, `cancelada` ou `falhou`

4.3 PONTO_DE_GRAVACAO

segments_N

Estado oficial do índice. Tudo que não está listado aqui não existe para os leitores.

CAMPO	TIPO	CHAVE	DESCRIÇÃO
`geracao`	vlong	PK	N do nome do arquivo; cresce a cada commit
`versao`	vlong		Contador de alterações
`contador`	vlong		Próximo número para nomear segmentos em base 36 (`_1` , `_a` , `_1z` ...)
`segmentos`	lista		Para cada segmento: nome, documentos, excluídos e geração de exclusão
`dados_usuario`	mapa texto→texto		Carimbo de tempo do commit e a última fonte indexada

4.4 SEGMENTO

`_N.si`

Mini-índice completo e imutável. Nasce de uma gravação do buffer em memória (origem `flush`) ou de uma mesclagem (`merge`).

CAMPO	TIPO	CHAVE	DESCRIÇÃO
`nome`	texto	PK	Prefixo de todos os arquivos do segmento
`geracao`	vlong	FK	Ponto de gravação que o publicou
`documentos`	vint		Total de doc_ids, incluindo os excluídos
`excluidos`	vint		Quantidade marcada no mapa de exclusões vigente
`geracao_exclusao`	vlong		G do arquivo `_N_G.liv` vigente; 0 quando não há exclusões
`origem`	texto		`flush` ou `merge`
`criado_em`	int64		Milissegundos Unix
`arquivos`	lista de texto		Os nove arquivos do segmento

4.5 CAMPO

`_N.si`

Esquema fixo de campos, gravado em cada segmento para que ele seja autodescritivo.

CAMPO	TIPO	CHAVE	DESCRIÇÃO
`numero`	vint	PK	0 id, 1 title, 2 body, 3 path, 4 ext, 5 lang, 6 source, 7 modified, 8 size, 9 description
`nome`	texto	UK	Nome usado na API e nas consultas (aceita apelidos em português, como `titulo` e `tipo`)
`opcoes_indice`	byte		0 não indexado, 1 só documentos, 2 com frequência, 3 com posições
`tem_normas`	bool		Verdadeiro para title, body e path, que entram no BM25 com normalização pelo tamanho
`doc_values`	byte		0 nenhum, 1 palavra-chave (ext, lang, source), 2 numérico (modified, size)

4.6 DOCUMENTO

_N.fdt e _N.fdx

Campos armazenados para mostrar o resultado e montar trechos. O corpo pode ser desligado com `Sonda:Indexing:StoreBody=false`; nesse caso, trechos, visualização, `/texto` e "Semelhantes" ficam sem conteúdo.

CAMPO	TIPO	CHAVE	DESCRIÇÃO
`segmento`, `doc_id`	texto, int32	PK	doc_id é a posição do documento no segmento
`fonte`	texto	FK	FONTE.nome
`chave`	texto	UK	`fonte:caminho`, indexada no campo `id`
`titulo`	texto		`<title>` do HTML, primeiro `#` do Markdown ou primeira linha do texto
`caminho`	texto		Caminho relativo à pasta da fonte, com barras normais
`tipo`, `idioma`	texto		Extensão sem ponto e código do idioma detectado (pt ou en)
`tamanho`, `modificado`	int64		Bytes do arquivo e data da última alteração em milissegundos Unix
`corpo`	texto		Texto extraído, usado nos trechos destacados, na visualização, no download e na busca por semelhantes

4.7 TERMO

_N.tim e _N.tip

Entrada do dicionário de um campo. O texto já passou por tokenização, minúsculas, remoção de stopwords, stemming e remoção de acentos.

CAMPO	TIPO	CHAVE	DESCRIÇÃO
`segmento`, `campo`, `texto`	texto, vint, utf-8	PK	Termos ordenados por bytes dentro de cada campo
`df`	vint		Quantidade de documentos que contêm o termo
`total_freq`	vlong		Soma das ocorrências (gravada como total_freq – df)
`ptr_doc`, `ptr_pos`	zlong		Início da lista em `.doc` e `.pos`, em delta dentro do bloco
`offset_saltos`	vlong		Distância até a tabela de saltos, só quando df ≥ 128
`doc_unico`	vint		Quando df = 1 o doc_id fica no próprio dicionário e nada é escrito em `.doc`
`freq_max`, `norma_min`	vint, byte		Maior frequência e menor tamanho de campo da lista, usados no limite superior do WAND

4.8 POSTAGEM e POSICAO

_N.doc e _N.pos

Lista invertida de um termo: os documentos em ordem crescente, quantas vezes o termo aparece e onde.

CAMPO	TIPO	CHAVE	DESCRIÇÃO
`termo`	referência	PK	TERMO dono da lista
`doc_id`	delta	PK	Diferença para o doc_id anterior; em blocos FOR de 128 e cauda em vint

CAMPO	TIPO	CHAVE	DESCRIÇÃO
`freq`	`vint`		Ocorrências no documento; nos blocos é gravada como freq – 1
`posicao`	`delta`		Posição do token no campo, contando também as stopwords descartadas

4.9 BLOCO_DE_SALTO

`_N.doc`

Tabela gravada depois da cauda de cada lista com 128 ou mais documentos. Permite pular blocos inteiros sem descompactar.

CAMPO	TIPO	CHAVE	DESCRIÇÃO
`termo`, `bloco`	`referência, int`	PK	Um registro por bloco cheio
`ultimo_doc`	`vint (delta)`		Maior doc_id do bloco; busca binária encontra o bloco do alvo
`fim_doc`, `fim_pos`	`vlong (delta)`		Posição do próximo bloco em `.doc` e `.pos`
`freq_max`, `norma_min`	`vint, byte`		Impacto máximo do bloco, para estimar a maior pontuação possível nele

4.10 NORMA

`_N.nrm`

Tamanho do campo em tokens, quantizado em 1 byte. Valores até 23 são exatos; acima disso a codificação é logarítmica e monotônica.

CAMPO	TIPO	CHAVE	DESCRIÇÃO
`segmento`, `campo`, `doc_id`	`texto, vint, int32`	PK	Uma linha por documento para title, body e path
`comprimento`	`byte`		Decodificado por tabela de 256 posições; o BM25 pré-calcula k1 × (1 – b + b × dl/avgdl) para cada byte

4.11 VALOR_DOCUMENTO

`_N.dv`

Valores colunares carregados na memória para facetas e ordenação sem ler os campos armazenados.

CAMPO	TIPO	CHAVE	DESCRIÇÃO
`segmento`, `campo`, `doc_id`	`texto, vint, int32`	PK	Uma linha por documento e por campo
`ordinal`	`int32`		Para ext, lang e source: posição no dicionário de valores do segmento (largura de 1, 2 ou 4 bytes)
`valor`	`texto | int64`		Para modified e size: mínimo do segmento + delta com 0, 1, 2, 4 ou 8 bytes

4.12 EXCLUSAO

`_N_G.liv`

Mapa de bits dos documentos excluídos. Cada commit que muda exclusões de um segmento grava uma geração nova.

CAMPO	TIPO	CHAVE	DESCRIÇÃO
`segmento`, `geracao`	texto, vlong	PK	G cresce a cada commit com exclusões novas
`tamanho`	vint		Quantidade de documentos do segmento
`bits`	int64[]		Bit 1 indica documento excluído

CAPÍTULO 05

Formato binário

Todos os arquivos começam com o mesmo cabeçalho e terminam com o mesmo rodapé. Entre eles fica o conteúdo próprio de cada formato.

5.1 Cabeçalho e rodapé

```
cabeçalho       int32 0x534F4E44 ("SOND")
                texto nome do formato (ex.: SondaPostingsDocs)
                int32 versão do formato (1)
                ...
rodapé          int32 ~0x534F4E44
                int32 0 (algoritmo: CRC32)
                int64 CRC32 de todos os bytes anteriores ao checksum
```

Ao abrir um segmento, os arquivos pequenos (`.si`, `.tip`, `.nrm`, `.dv`, `.liv`, `.fdx` e `segments_N`) têm o checksum conferido na hora. Os grandes são conferidos sob demanda pelo comando `sonda verificar` ou pelo botão **Verificar integridade** do painel (também disponível na paleta de comandos).

5.2 Dicionário de termos (.tim e .tip)

```
.tim            por campo, blocos de até 32 termos:
                   vint quantidade de termos no bloco
                   por termo:
                      vint bytes compartilhados com o termo anterior
                      vint bytes novos + bytes novos
                      vint df
                      vlong total_freq − df (campos com frequência)
                      vint doc_unico | zlong delta ptr_doc
                      zlong delta ptr_pos (campos com posições)
                      vlong offset_saltos (df ≥ 128)
                      vint freq_max + byte norma_min (campos com frequência)

.tip            vint quantidade de campos
                   por campo: vint número, vlong termos, vlong soma df, vlong soma total_freq, vint
                   documentos com o campo, vint blocos,
                      por bloco: texto primeiro termo, vlong delta do ponteiro
```

O `.tip` inteiro fica na memória. Para achar um termo, o leitor faz busca binária nos primeiros termos dos blocos, posiciona no bloco certo do `.tim` e lê no máximo 32 entradas.

5.3 Postagens (.doc e .pos)

```
.doc            por termo com df > 1:
                   blocos cheios (df / 128 vezes):
                      bloco FOR 128 deltas de doc_id
                      bloco FOR 128 valores freq − 1
                   cauda (df mod 128 documentos):
                      vint (delta << 1) | (freq == 1) e vint freq se freq ≠ 1
                   tabela de saltos (df ≥ 128):
                      vint quantidade
                      por bloco: vint Δultimo_doc, vlong Δfim_doc, vlong Δfim_pos, vint freq_max,
                      byte norma_min
.pos            por documento da lista: freq valores vint com o delta da posição
```

Um bloco FOR guarda um byte com a largura em bits e depois os 128 valores empacotados. Se os deltas cabem em 5 bits, o bloco ocupa 81 bytes em vez de 512.

5.4 Campos armazenados (.fdt e .fdx)

```
.fdt            blocos de até 128 documentos ou 48 KB:
                   vint doc inicial, vint documentos, vint bytes originais, vint bytes comprimidos
                   bytes comprimidos com Brotli (qualidade 4, janela 22)
                   dentro do bloco, por documento:
                      vint campos, e por campo: vint número, byte tipo (0 texto, 1 int64), valor
.fdx            vint documentos, vint blocos, por bloco: vint Δdoc inicial, vlong Δponteiro
```

Ler um documento custa uma busca binária no `.fdx` (na memória), uma leitura e uma descompressão. O último bloco descomprimido fica em cache por thread.

5.5 Normas, doc values, exclusões e ponto de gravação

```
.nrm            vint documentos, vint campos, por campo: vint número + documentos × byte
.dv             vint documentos, vint campos, por campo:
                   palavra-chave: vint valores, textos, byte largura, documentos × (ordinal + 1)
                   numérico: zlong mínimo, byte largura, documentos × delta
.liv            vint tamanho, vint palavras, palavras × int64
segments_N      vlong geração, vlong versão, vlong contador, vint segmentos,
                   por segmento: texto nome, vint documentos, vint excluídos, vlong geração_exclusao
                   vint pares, pares de textos (dados do usuário)
```

CAPÍTULO 06

Regras de consistência

Como o índice continua correto com várias threads indexando, buscas acontecendo ao mesmo tempo e o processo podendo cair a qualquer momento.

6.1 Gravação (commit)

1. O buffer em memória é congelado e trocado por um novo em microssegundos, sob um `ReaderWriterLockSlim`. As threads continuam indexando no buffer novo.
2. O buffer congelado é gravado como segmento: campos armazenados, normas, doc values, postagens, posições, dicionário e por fim o `.si`. Cada arquivo é sincronizado no disco (`Flush(true)`).
3. Para cada segmento com exclusões novas, um `_N_G.liv` com geração nova é gravado.
4. O `pending_segments_N` é gravado, sincronizado e renomeado para `segments_N`. A renomeação é o momento atômico em que o novo estado passa a valer.
5. Arquivos que nenhum ponto de gravação, segmento vivo ou mesclagem em andamento usa são apagados. No Windows, um arquivo ainda aberto por um leitor entra numa fila e é apagado depois.

Se o processo cair antes do passo 4, o índice reabre no ponto de gravação anterior e os arquivos órfãos são apagados na próxima abertura.

6.2 Exclusões e atualizações

- Atualizar é excluir a chave em todos os segmentos e no buffer, e depois incluir o documento novo.
- A exclusão marca o bit no mapa em memória do segmento; leitores só veem a mudança depois do próximo commit.
- Uma chave incluída duas vezes no mesmo buffer marca a versão anterior como excluída já na gravação do segmento.

6.3 Mesclagem

- Quando há 10 segmentos (configurável), os menores são mesclados num só. Segmentos com 30% ou mais de excluídos também são reescritos.
- A mesclagem faz uma intercalação de k vias dos dicionários, renumera os doc_ids pulando os excluídos e reconstrói postagens, posições, normas, doc values e campos armazenados.
- Exclusões que chegam durante a mesclagem são remapeadas para o segmento novo antes da publicação, que acontece junto com um commit.
- Buscas abertas continuam usando os segmentos antigos até terminarem; a contagem de referências fecha os arquivos só depois disso.

6.4 Leitura concorrente

- Cada busca pega uma referência ao leitor atual (snapshot imutável) e devolve ao terminar. Exportações, feeds, sugestões e “Semelhantes” usam o mesmo caminho de leitura e nunca bloqueiam a indexação.
- Leituras usam `RandomAccess.Read` com posição explícita, então várias threads leem o mesmo arquivo sem travas.
- Estatísticas de coleção (N, df e tamanho médio) são somadas entre segmentos, para que a pontuação não dependa de como os documentos foram divididos.

CAPÍTULO 07

Dados fora do índice

A versão 1.4.0 acrescenta dados que não são gravados nos segmentos: estatísticas de busca em memória, o corpo dos webhooks e os formatos de exportação. Nenhum deles altera o formato do índice.

7.1 ESTATISTICA_BUSCA

memória do servidor · GET /api/buscas/estatisticas · /metrics

Cada chamada a `/api/busca` registra uma linha num anel com as últimas 500 buscas. O anel vive só na memória do processo: some ao reiniciar o servidor e não vai para o catálogo nem para o índice. A partir dele o servidor calcula os totais, a média e o p95 de latência, as consultas mais frequentes e as que não acharam nada (painel **Consultas** da Visão geral).

CAMPO	TIPO	DESCRIÇÃO
`consulta`	texto	Consulta como foi digitada; agrupada sem diferenciar maiúsculas para os rankings
`total`	int64	Resultados encontrados; 0 entra em `semResultados`
`ms`	double	Tempo da busca em milissegundos, usado na média e no p95
`momento`	data UTC	Quando a busca foi feita

RESPOSTA DE GET /API/BUSCAS/ESTATISTICAS

```
{
  "total": 128, "semResultado": 9, "mediaMs": 2.4, "p95Ms": 6.1,
  "principais":    [{ "consulta": "índice invertido", "vezes": 14, "ultimoTotal": 8 }],
  "semResultados": [{ "consulta": "kubernetes", "vezes": 3 }],
  "recentes":      [{ "consulta": "farol", "total": 3, "ms": 1.8,
                      "momento": "2026-10-27T14:02:11Z" }]
}
```

7.2 EVENTO_WEBHOOK

POST para cada URL de Sonda:Webhooks:Urls

Quando um evento listado em `Sonda:Webhooks:Events` acontece, o servidor envia um POST em segundo plano para cada URL configurada. Falhas e tempo esgotado (`TimeoutSeconds`, 10 s por padrão) só aparecem no log: o envio nunca atrasa a indexação nem a busca, e nada é guardado para reenvio.

CAMPO OU CABEÇALHO	TIPO	DESCRIÇÃO
`evento`	texto	`execucao` (fim de uma indexação, com os mesmos campos de EXECUCAO_INDEXACAO) ou `indice` (novo ponto de gravação publicado)
`momento`	data UTC	Quando o evento aconteceu, em ISO 8601
`dados`	objeto	Conteúdo do evento, igual ao enviado por `/api/eventos`
`X-Sonda-Evento`	cabeçalho	Nome do evento, para rotear sem ler o corpo

CAMPO OU CABEÇALHO	TIPO	DESCRIÇÃO
`X-Sonda-Assinatura`	cabeçalho	Só com `Secret`: `sha256=` + HMAC-SHA256 do corpo em hexadecimal

CORPO DO EVENTO EXECUCAO

```
{
  "evento": "execucao",
  "momento": "2026-10-27T14:05:42Z",
  "dados": { "id": "k3f9a1c0b2de", "fonte": "exemplos", "modo": "Incremental",
             "encontrados": 30, "novos": 0, "atualizados": 1, "removidos": 0,
             "falhas": 0, "situacao": "concluida" }
}
```

7.3 Formatos de exportação

`GET /api/busca/exportar?formato=csv|json|ndjson · GET /api/busca/feed`

A exportação roda a mesma busca do painel, com os mesmos filtros, e devolve até `limite` resultados (500 por padrão, no máximo 1.000) como anexo `sonda-busca.csv`, `.json` ou `.ndjson`. Os campos são os mesmos nos três formatos:

COLUNA	TIPO	DESCRIÇÃO
`posicao`	int	Ordem no ranking, a partir de 1
`pontuacao`	double	Pontuação BM25 ou TF-IDF
`titulo`	texto	DOCUMENTO.titulo
`fonte`, `caminho`	texto	Partes da chave do documento
`tipo`, `idioma`	texto	Extensão e idioma detectado
`tamanho`	int64	Bytes do arquivo
`modificado`	data UTC	Última alteração do arquivo
`trecho`	texto	Trecho destacado em texto puro, sem marcação

- **CSV:** UTF-8 com BOM (abre direto no Excel), separador vírgula e aspas duplas. Células de texto que começam com `=`, `+`, `-`, `@`, tabulação ou retorno de carro recebem um apóstrofo na frente, para não virarem fórmula.
- **JSON:** um envelope `{ consulta, total, exportados, geradoEm, resultados }`, com um objeto por resultado em `resultados`. **NDJSON:** um objeto por linha, sem envelope, bom para `jq`, Logstash e cargas em lote.
- **Feed Atom:** os 20 primeiros resultados como `application/atom+xml`, cada entrada com o título, o trecho e um link para `/#/?q=…` no painel.

CAPÍTULO 08

Como criar e manter

O índice é criado na primeira execução e não precisa de nenhum script SQL.

8.1 Criando o índice

1. Rode `iniciar.cmd` (Windows) ou `./iniciar.sh` (Linux e macOS). A pasta `dados/indice` é criada e os exemplos são indexados.
2. Ou, pela linha de comando: `dotnet run --project src/Sonda.Cli -- indexar C:\Documentos --fonte documentos`.
3. Confira com `sonda estatisticas` ou na tela **Visão geral** do painel.

8.2 Backup e restauração

Com o servidor parado, copie a pasta `dados/` inteira. Com o servidor rodando, copie logo depois de um commit: os segmentos são imutáveis e o `segments_N` mais novo descreve um estado completo. Um webhook do evento `indice` é um bom gatilho para um backup automático.

Para restaurar, pare o servidor, substitua a pasta e inicie de novo. Rode `sonda verificar` para conferir os checksums. As estatísticas de busca não entram no backup, porque vivem só na memória.

8.3 Evoluindo o formato

Cada formato tem nome e versão no cabeçalho. Uma versão nova deve aceitar ler a anterior ou exigir reindexação completa. A 1.4.0 mantém a versão 1 de todos os formatos.

Como os documentos originais continuam nas pastas das fontes, reindexar do zero é sempre possível: apague `dados/indice` e rode a indexação completa.